



30-Day Android Challenge

Mastering Android Programming Interviews Globally
(Junior-Mid-Senior-Architect-Level Deconstruction 📱)

Day 9:

Testing Strategies

(Unit, Integration, UI Testing)

Part 1 of 2



Esmaeeil Moradi

@esmaeeil-moradi700

Day 9

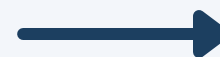


of #30DayAndroidChallenge: Testing Strategies (Unit, Integration, UI Testing)

Hello! I'm Esmaeeil Moradi, a Senior Android Architect with over 10 years of hands-on experience in Android development (including AOSP), Kotlin, and specialized expertise in Web3. For Day 9 of the #30DayAndroidChallenge, we delve into the critical topic of "Testing Strategies in Android." This subject forms the backbone of quality assurance in any professional application, and I intend to provide insights that will not only prepare you for top-tier interviews but also empower you to build reliable and robust applications in the real world. "Ensuring Application Quality Through Comprehensive Testing" is our motto for today.



Esmaeeil Moradi
@esmaeeil-moradi700



Junior Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700

What is the difference between Unit Testing and UI Testing?



Tests in software development are used to ensure that the application functions correctly and as expected. Unit testing and UI testing are two fundamental types of tests that have different objectives and methodologies.



Esmaeel Moradi

@esmaeel-moradi700

Unit Testing:



- **Goal:** To verify the smallest logical "unit" of code (a function, a method, a class) in isolation. The aim is to ensure that each part of the code works correctly, independently of other parts.
- **Focus:** Pure business logic, algorithms, utility functions, or classes that have no direct dependency on the Android framework or the user interface.
- **Execution Speed:** Very fast. They typically run on the JVM on the developer's machine and do not require an emulator or physical device.
- **Coverage:** Provides broad code coverage, as it can test the most granular parts.
- **Complexity:** Relatively simple to write.
- **Example:** Testing a function that sorts a list, or a class that performs mathematical calculations.



Esmaeeil Moradi
@esmaeeil-moradi700

UI Testing:

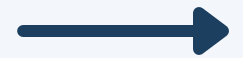


- **Goal:** To verify user interactions with the application's user interface and ensure that the UI displays correctly, responds to user input, and user flows behave as expected.
- **Focus:** The entire application or large parts of it, including Activities, Fragments, Composables, Views, and how they interact with each other.
- **Execution Speed:** Relatively slow. They require an emulator or physical device to render the UI and simulate interactions.
- **Coverage:** Less code coverage (only paths accessible via the UI) but provides a better understanding of the user experience.
- **Complexity:** Generally more complex to write and maintain (due to dependencies on UI state and timing).
- **Example:** Testing that clicking a "Login" button navigates to the home screen; or that invalid input in a field displays the appropriate error.



Esmaeel Moradi
@esmaeeil-moradi700

Summary of Differences:



Feature	Unit Testing	UI Testing
Main Goal	Validate the smallest logical unit of code	Validate user interaction with UI and user flows
Isolation	Fully isolated, dependencies are mocked	Requires larger parts or the entire system
Execution Env.	JVM (no Android device)	Emulator/Android device (needs Instrumentation)
Speed	Fast	Slow
Complexity	Low	High
Code Coverage	High	Relatively Low

Both types of tests are essential for building a quality application and are complementary to each other.



Esmaeel Moradi
@esmaeel-moradi700

How to write a simple Unit Test in Kotlin using JUnit?

JUnit is a very popular testing framework for Java and Kotlin. To write a simple unit test, we follow these steps:

1. Add Dependencies to build.gradle.kts (in the app module or relevant logic module):

```
// app/build.gradle.kts (or the logic module's build.gradle.kts)

dependencies {
    // For local unit tests that run on the JVM
    testImplementation("junit:junit:4.13.2")
    testImplementation("org.jetbrains.kotlin:kotlin-test-junit:1.9.0") // Or your Kotlin version
    testImplementation("org.mockito.kotlin:mockito-kotlin:5.2.1") // Or your MockK version if preferred
    // ... other dependencies
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

2. Create a Class to Test:



Unit tests are typically placed in the src/test/java (or src/test/kotlin) folder. Let's assume we have a Calculator class that we want to test:

```
// src/main/java/com/example/myapp/Calculator.kt
package com.example.myapp

class Calculator {
    fun add(a: Int, b: Int): Int {
        return a + b
    }

    fun subtract(a: Int, b: Int): Int {
        return a - b
    }
}
```



Now, create the test class in
src/test/java/com/example/myapp/CalculatorTest.kt:

```
// src/test/java/com/example/myapp/CalculatorTest.kt
package com.example.myapp

import org.junit.Assert.assertEquals
import org.junit.Before
import org.junit.Test

class CalculatorTest {

    private lateinit var calculator: Calculator

    // @Before: This method runs once before each test (@Test method).
    // Suitable for initial setup and initialization required for the tests.
    @Before
    fun setup() {
        calculator = Calculator()
    }

    // @Test: This annotation indicates that the following method is a test.
    @Test
    fun add_positiveNumbers_returnsCorrectSum() {
        // Given (Arrangement): Inputs and test conditions
        val a = 5
        val b = 3

        // When (Action): Calling the method under test
        val result = calculator.add(a, b)

        // Then (Assertion): Verifying the result using Assertions
        assertEquals(8, result) // Expect 5 + 3 to be 8
    }

    @Test
    fun subtract_positiveNumbers_returnsCorrectDifference() {
        // Given
        val a = 10
        val b = 4

        // When
        val result = calculator.subtract(a, b)

        // Then
        assertEquals(6, result) // Expect 10 - 4 to be 6
    }
}
```



Esmaeel Moradi
@esmaeeil-moradi700

Code Explanation:

- @Before: A method that runs before each test method. We used it to initialize calculator.
- @Test: This annotation indicates that the add_positiveNumbers_returnsCorrectSum() method is a test method that JUnit will execute.
- assertEquals(expected, actual): This is an Assertion from JUnit that checks whether the actual value (the real result) is equal to the expected value (the anticipated result). If they are not equal, the test fails.

Running the Test:

You can right-click on the green icon next to the CalculatorTest class or next to any test method in Android Studio and select "Run" to execute the tests. The results will be displayed in the "Run" window.

This is a basic example of a unit test. In more complex scenarios, we use Mocking (explained in subsequent questions) to isolate dependencies.



Esmaeel Moradi

@esmaeeil-moradi700

Name a tool for UI Testing in Android.



The primary and official tool for UI testing in Android is Espresso.

Espresso:

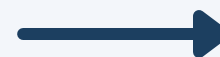
Espresso is a lightweight, API-based testing framework provided by Google, used for testing the user interface of Android applications. It allows developers to simulate user interactions and verify that the UI behaves as expected. Espresso automatically waits for UI operations (like animations or data loading) to complete, which makes it reliable and less "Flaky" for UI tests.

For Jetpack Compose, the corresponding tool is Compose Test APIs, which provides similar capabilities to Espresso but for UIs written with Jetpack Compose. These APIs are part of the `androidx.compose.ui:ui-test` library.



Esmaeel Moradi

@esmaeel-moradi700



Mid-Level Questions



Esmaeel Moradi
@esmaeeil-moradi700

How to write an Instrumented Test using Espresso?

Provide an example for testing a button click.

Instrumented Tests, unlike unit tests, run on an actual Android device or emulator and can access Android framework APIs (like Context, Activity, View). Espresso is the main tool for writing these types of tests for View-based UIs (not Compose).

1. Add Dependencies to build.gradle.kts (in the app module):

```
// app/build.gradle.kts

dependencies {
    // For instrumented tests that run on a device or emulator
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
    androidTestImplementation("androidx.test:runner:1.5.2")
    androidTestImplementation("androidx.test:rules:1.5.0") // Needed for ActivityScenarioRule
    // If you're using Compose UI, you'll also need:
    androidTestImplementation(platform("androidx.compose:compose-bom:2023.08.00")) // Or 1
    androidTestImplementation("androidx.compose.ui:ui-test-junit4")
    debugImplementation("androidx.compose.ui:ui-test-manifest") // For compose test manifest
}
```



Esmaeel Moradi
@esmaeeil-moradi700

2. Create a Simple Activity (e.g., MainActivity.kt):

Suppose our MainActivity contains a Button and a TextView. When the button is clicked, the TextView's text changes.

```
// src/main/java/com/example/myapp/MainActivity.kt
package com.example.myapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private lateinit var messageTextView: TextView
    private lateinit var changeTextButton: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        messageTextView = findViewById(R.id.message_text_view)
        changeTextButton = findViewById(R.id.change_text_button)

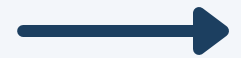
        changeTextButton.setOnClickListener {
            messageTextView.text = "Hello Espresso!"
        }
    }
}
```



Esmaeel Moradi

@esmaeel-moradi700

3. XML Layout File (e.g., activity_main.xml):



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <TextView
        android:id="@+id/message_text_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Initial Text"
        android:textSize="24sp"
        android:padding="16dp"/>

    <Button
        android:id="@+id/change_text_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Change Text"
        android:padding="16dp"/>

</LinearLayout>
```



Esmaeel Moradi
@esmaeel-moradi700

4. Write the Espresso Test in

src/androidTest/java/com/example/myapp/
MainActivityEspressoTest.kt:



```
// src/androidTest/java/com/example/myapp/MainActivityEspressoTest.kt
package com.example.myapp

import androidx.test.espresso.Espresso.onView
import androidx.test.espresso.action.ViewActions.click
import androidx.test.espresso.assertion.ViewAssertions.matches
import androidx.test.espresso.matcher.ViewMatchers.withId
import androidx.test.espresso.matcher.ViewMatchers.withText
import androidx.test.ext.junit.rules.ActivityScenarioRule
import androidx.test.ext.junit.runners.AndroidJUnit4
import org.junit.Rule
import org.junit.Test
import org.junit.runner.RunWith

// @RunWith: Runs JUnit with AndroidJUnit4 Runner, essential for instrumented tests.
@RunWith(AndroidJUnit4::class)
class MainActivityEspressoTest {

    // @Rule: This Rule launches the specified Activity before each test
    // and correctly cleans it up after the test completes.
    @get:Rule
    val activityRule = ActivityScenarioRule(MainActivity::class.java)

    @Test
    fun buttonClick_changesTextViewText() {
        // 1. Given (Arrangement):
        // Our Activity is launched with "Initial Text" in the TextView.
        // onView(withId(R.id.message_text_view))
        // .check(matches(withText("Initial Text"))) // Optional: Verify initial state

        // 2. When (Action):
        // Find the button with ID "change_text_button" and simulate a click on it.
        onView(withId(R.id.change_text_button))
            .perform(click())

        // 3. Then (Assertion):
        // Find the TextView with ID "message_text_view"
        // and assert that its text has changed to "Hello Espresso!".
        onView(withId(R.id.message_text_view))
            .check(matches(withText("Hello Espresso!")))
    }
}
```



Esmaeel Moradi

@esmaeeil-moradi700

Espresso Code Explanation:



- `@get:Rule val activityRule = ActivityScenarioRule(MainActivity::class.java)`: This line creates a JUnit "Rule" that launches MainActivity before each test. This Rule ensures your activity is in a clean state for each test.
- `onView()`: Espresso's main method for finding Views in the UI hierarchy.
- `withId(R.id.change_text_button)`: A Matcher for finding a View with a specific ID.
- `withText("Expected Text")`: Another Matcher for finding a View with specific text.
- `.perform()`: A method for performing a ViewAction on the found View.
- `click()`: A ViewAction that simulates a click on the View.
- `.check()`: A method for performing a ViewAssertion on the found View.
- `matches()`: A ViewAssertion that checks if the View matches a specific Matcher.
- `ViewAssertions.matches(ViewMatchers.withText("Hello Espresso!"))`: This line checks that the found View (TextView) contains the text "Hello Espresso!".



Esmaeel Moradi

@esmaeeil-moradi700



Execution:

Espresso tests run on an emulator or a real device. You can run the test by right-clicking on the test class or test method in Android Studio and selecting "Run".



Esmaeel Moradi
@esmaeeil-moradi700

Explain the use of Mockito or MockK for Mocking dependencies in Unit Tests.

In unit tests, the main goal is to isolate the Unit Under Test (UUT) from its external dependencies. This means that if our UUT depends on other classes, network services, a database, or even Android's Context, we don't want these real dependencies to be active during the unit test. The reasons for this are:

1. **Speed:** Real dependencies (like network calls) make tests very slow.
2. **Determinism:** A test should produce the same result every time it runs. Real dependencies can lead to unpredictable results (e.g., network is down).
3. **Isolation:** If a test fails, we want to quickly identify that the problem is within our code unit, not in one of its dependencies.

This is where Mocking comes in. Mocking means creating a "mock" or "simulated" version of real dependencies. These mocks have predefined behavior that we specify in the test, without actually performing the real operation.



Esmaeel Moradi
@esmaeeil-moradi700

Mockito and MockK:



- **Mockito:** A popular mocking framework for Java, widely used in Kotlin projects as well.
- **MockK:** A dedicated mocking framework for Kotlin that better supports Kotlin language features (like Extension Functions, Top-level Functions, and Object/Companion Objects) and has a more fluent and less verbose syntax.

Usage Scenario: Suppose we have a **UserRepository** class responsible for fetching user information from an API, and a **UserViewModel** that uses **UserRepository** to display user information.



Esmaeel Moradi

@esmaeeil-moradi700

```

// src/main/java/com/example/myapp/UserRepository.kt
package com.example.myapp

// A simple data class
data class User(val id: String, val name: String, val email: String)

// Interface for repository, good practice for testability
interface UserRepository {
    suspend fun fetchUser(userId: String): User
    suspend fun saveUser(user: User): Boolean
}

// Actual implementation (would involve network calls)
class RealUserRepository : UserRepository {
    override suspend fun fetchUser(userId: String): User {
        // Simulate network delay and data fetching
        kotlinx.coroutines.delay(1000)
        if (userId == "1") {
            return User("1", "Esmaeil", "esmaeil@example.com")
        }
        throw Exception("User not found")
    }

    override suspend fun saveUser(user: User): Boolean {
        // Simulate saving to DB/network
        kotlinx.coroutines.delay(500)
        return true
    }
}

// src/main/java/com/example/myapp/UserViewModel.kt
package com.example.myapp

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

class UserViewModel(private val userRepository: UserRepository) : ViewModel() {

    private val _userState = MutableStateFlow<User?>(null)
    val userState: StateFlow<User?> = _userState

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading

    private val _errorMessage = MutableStateFlow<String?>(null)
    val errorMessage: StateFlow<String?> = _errorMessage

    fun loadUser(userId: String) {
        viewModelScope.launch {
            _isLoading.value = true
            _errorMessage.value = null
            try {
                val user = userRepository.fetchUser(userId)
                _userState.value = user
            } catch (e: Exception) {
                _errorMessage.value = e.message
                _userState.value = null
            } finally {
                _isLoading.value = false
            }
        }
    }
}

```





Testing UserViewModel with Mockito-Kotlin:

To test UserViewModel, we don't want actual network calls to happen. So, we'll mock the UserRepository.

1. Dependencies:

```
// app/build.gradle.kts
dependencies {
    // ...
    testImplementation("org.mockito.kotlin:mockito-kotlin:5.2.1")
    testImplementation("org.mockito:mockito-inline:5.2.0") // Needed for mocking final classes/object
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.0") // For testing coroutines
    // ...
}
```



Esmaeel Moradi

@esmaeeil-moradi700

2. Write UserViewModelTest in src/test/java/com/example/myapp/UserViewModelTest.kt:



```
// src/test/java/com/example/myapp/UserViewModelTest.kt
package com.example.myapp

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.ExperimentalCoroutinesApi
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.test.StandardTestDispatcher
import kotlinx.coroutines.test.advanceUntilIdle
import kotlinx.coroutines.test.resetMain
import kotlinx.coroutines.test.runTest
import kotlinx.coroutines.test.setMain
import org.junit.After
import org.junit.Before
import org.junit.Test
import org.mockito.kotlin.mock
import org.mockito.kotlin.verify
import org.mockito.kotlin.whenever
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue

@OptIn(ExperimentalCoroutinesApi::class)
class UserViewModelTest {

    private lateinit var userRepository: UserRepository
    private lateinit var userViewModel: UserViewModel
    private val testDispatcher = StandardTestDispatcher() // For controlling coroutine execution

    @Before
    fun setup() {
        // 1. Mock the dependency
        userRepository = mock() // Create a mock instance of UserRepository
        userViewModel = UserViewModel(userRepository)

        // Set the main dispatcher for testing coroutines
        Dispatchers.setMain(testDispatcher)
    }

    @After
    fun tearDown() {
        // Reset the main dispatcher after each test
        Dispatchers.resetMain()
    }
}
```



Esmaeel Moradi

@esmaeeil-moradi700



```
@Test
fun loadUser_success_updatesUserStateAndLoadingStatus() = runTest {
    // Given: Define the behavior of the mock
    val expectedUser = User("1", "Test User", "test@example.com")
    whenever(userRepository.fetchUser("1")).thenReturn(expectedUser) // When fetch

    // When: Call the method under test
    userViewModel.loadUser("1")

    // Advance coroutines until all pending coroutines are finished
    advanceUntilIdle()

    // Then: Verify the state of the ViewModel
    assertFalse(userViewModel.isLoading.first()) // Loading should be false
    assertNull(userViewModel.errorMessage.first()) // No error message
    assertEquals(expectedUser, userViewModel.userState.first()) // User state should

    // Verify that the mock method was called
    verify(userRepository).fetchUser("1")
}

@Test
fun loadUser_failure_updatesErrorMessageAndLoadingStatus() = runTest {
    // Given: Define the behavior of the mock for a failure scenario
    val errorMessage = "Network Error"
    whenever(userRepository.fetchUser("2")).thenThrow(RuntimeException(errorMessage))

    // When
    userViewModel.loadUser("2")
    advanceUntilIdle()

    // Then
    assertFalse(userViewModel.isLoading.first())
    assertEquals(errorMessage, userViewModel.userState.first()?.name) // This asse
    assertNull(userViewModel.userState.first())
    assertEquals(errorMessage, userViewModel.errorMessage.first())

    verify(userRepository).fetchUser("2")
}
}
```



Esmaeel Moradi

@esmaeeil-moradi700

Mocking Explanation:

- `mock()`: A method from Mockito-Kotlin to create a mock object of `UserRepository`. This object has all the methods of `UserRepository` but no real execution logic.
- `whenever(userRepository.fetchUser("1")).thenReturn(expectedUser)`: This line defines the "behavior" of the mock. We tell Mockito, "When the `fetchUser` method is called on `userRepository` with the argument "1", return `expectedUser`."
- `whenever(...).thenThrow(...)`: For error scenarios, we can specify that the mock should throw an exception.
- `verify(userRepository).fetchUser("1")`: This method from Mockito is used for "verification". We can verify whether a specific method was called on the mock. This is useful for ensuring the UUT correctly interacts with its dependencies.
- Kotlin Coroutines Test: Using `StandardTestDispatcher` and `runTest` from the `kotlinx-coroutines-test` library is crucial for precise control over Coroutine execution in unit tests. `advanceUntilIdle()` ensures that all pending Coroutines are completed before validating the final state.



Esmaeel Moradi

@esmaeeil-moradi700



Benefits of Mocking:

- **Testability:** Classes designed for mocking (typically through Interfaces or Dependency Injection) are easier to test.
- **High Test Coverage:** Allows different code paths (e.g., success, failure, error scenarios) to be easily tested.
- **Speed:** Tests run quickly because no real I/O operations (like network or disk) occur.

Describe the Testing Pyramid and its relevance to Android development.

The Testing Pyramid is a conceptual model introduced by Mike Cohn that provides a strategic approach to organizing different types of tests in software development. This pyramid suggests that certain types of tests should exist in larger quantities (at the base of the pyramid) and other types in smaller quantities (at the top).



Esmaeeil Moradi

@esmaeeil-moradi700

Testing Pyramid Structure:



1-Base (Bottom of the Pyramid): Unit Tests

- Quantity: Very many.
- Speed: Fast.
- Cost: Low (to write and maintain).
- Goal: To test the smallest unit of code (functions, classes). These tests are fully isolated, and their dependencies are mocked.
- In Android: Testing ViewModels, Use Cases, Repositories (without Android API dependencies), complex algorithms, and any pure business logic. These tests run on the JVM in src/test/java.



Esmaeel Moradi

@esmaeel-moradi700

2-Middle (Middle of the Pyramid): Integration Tests

- Quantity: Medium.
- Speed: Medium.
- Cost: Medium.
- Goal: To verify the interaction between multiple code units or different components (e.g., ViewModel with Repository, or a service with a local database). These tests have less isolation and might use less mocking or involve real parts of the system.
- In Android:
 - Instrumented Tests: Testing interactions between Activities/Fragments and ViewModels, or testing Compose components with ViewModel (using composeTestRule).
 - Repository tests with a real database (Room/Realm): Testing that data is correctly stored and retrieved.
 - Background service tests: Testing that services correctly interact with the system or other components.
 - These tests typically run in src/androidTest/java and require an emulator or device.



Esmaeel Moradi

@esmaeel-moradi700

3-Apex (Top of the Pyramid): UI Tests / End-to-End (E2E) / Acceptance Tests

- Quantity: Few.
- Speed: Slow.
- Cost: High (to write and maintain).
- Goal: To test the entire system from a user's perspective, simulating real user flows. These tests involve the UI, business logic, database, and network (or mocked). They typically test key "happy path" or critical scenarios.
- In Android:
 - Espresso tests: Full user flow testing in Activities/Fragments.
 - Compose UI Tests: Full user flow testing in Compose (with `composeTestRule`).
 - Automated UI tests that run on a device or emulator.
 - May include tests that interact with a real or mocked backend (using tools like `MockWebServer`).



Esmaeel Moradi

@esmaeel-moradi700



Relevance of the Testing Pyramid to Android Development:

The Testing Pyramid is an excellent guide for Android teams to optimize testing resources:

- Benefits of many Unit Tests:
 - Fast Feedback: Developers quickly find out if their changes broke anything.
 - Easy Debugging: Finding the source of bugs in an isolated unit is much simpler.
 - High Coverage: Ensures that most of the code logic is covered.
 - Code Health: Encourages writing modular and testable code.
- Benefits of medium Integration Tests:
 - Ensuring correct collaboration between different parts of the code.
 - Identifying issues across layers (e.g., Data Layer with Domain Layer).
- Benefits of few UI Tests:
 - UI tests are the most expensive and flaky tests. Keep them only for critical and essential user scenarios.
 - Ensuring the final user experience works correctly.



Esmaeel Moradi
@esmaeel-moradi700

Conclusion:

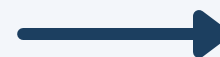


By following the testing pyramid, Android teams can create an efficient and effective testing strategy that strikes a good balance between speed, cost, and ensuring application quality. This approach is particularly crucial in complex and large applications, such as a cryptocurrency wallet, where reliability and security are of paramount importance.



Esmaeel Moradi

@esmaeel-moradi700



Senior Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700

Design a comprehensive testing strategy for a complex Android application, including unit, integration, and UI tests.

For a complex Android application (e.g., a banking app, messaging app, or crypto wallet), the testing strategy must be multi-faceted, layered, and proportional to the importance of each section. The goal of this strategy is to achieve reliable test coverage while maintaining speed and efficiency in the development process.



Esmaeeil Moradi
@esmaeeil-moradi700

Comprehensive Testing Strategy:

Level 1: Unit Tests - Base of the Pyramid (70-90% Test Coverage):

- Execution Location: JVM (src/test/java folder).
- Goal: Testing pure business logic, Use Cases (Interactors), ViewModels, Utility Classes, and any class that has no direct dependency on the Android framework.
- Tools: JUnit 5 (or 4), Mockito-Kotlin/MockK (for mocking Repositories, Services, and other dependencies), Kotlinx Coroutines Test (for testing asynchronous code).
- Best Practices:
 - Full Isolation: All external dependencies (network, database, UI) must be mocked.
 - Speed: Tests should run within milliseconds to provide immediate feedback to the developer.
 - High Coverage: Focus on covering all code paths, including success, failure, error, and edge cases.
 - Test-Driven Development (TDD): If possible, use the TDD approach to write more testable code.

Level 2: Integration Tests - Middle of the Pyramid (10-20% Test Coverage):

- Execution Location: Emulator/device (src/androidTest/java folder or src/test/java for local integrations).



Esmaeel Moradi
@esmaeel-moradi700



- Goal:
 - Inter-layer Interaction: Testing that the ViewModel correctly communicates with the Repository, or that the Repository correctly interacts with the database/API.
 - Android Components: Testing Fragments/Composables in isolation (e.g., testing a Composable with a real or mocked ViewModel).
 - Database: Testing Room (or other ORM) with an in-memory database or a real test database.
 - Network: Testing the network layer with a Mock Web Server (like OkHttp's MockWebServer) that returns predefined responses.
- Tools: AndroidX Test (JUnit 4/5 for Android), MockWebServer, Hilt/Koin Test (for testing with DI), Compose Test Rule (for testing Composables).
- Best Practices:
 - Focus on Boundaries: Testing points where two or more components interact.
 - Use Mocks for More External Dependencies: For example, in a database and Repository integration test, the network might be mocked.
 - Careful Scenario Selection: Testing scenarios where individual units cannot identify bugs on their own.
 - Maintain Reasonable Speed: Despite the need for a device/emulator, try to keep tests as fast as possible.



Esmaeel Moradi
@esmaeel-moradi700

Level 3: UI Tests / End-to-End Tests - Apex of the Pyramid (5-10% Test Coverage):

- Execution Location: Emulator/device (src/androidTest/java folder).
- Goal: Simulating complete critical user journeys and vital scenarios from the end-user's perspective. These tests involve all layers (UI, ViewModel, Repository, Network (can be mocked), Database).
- Tools: Espresso (for View System), Compose Test APIs (for Jetpack Compose), UI Automator (for inter-app or out-of-app interactions), MockWebServer (for controlling network responses in tests).
- Best Practices:
- Happy Path Scenarios: Focus on the most important successful user paths.
- Minimize Flakiness: Correctly use Espresso/Compose Test APIs synchronization capabilities (like waitUntil, advanceUntilIdle) and ensure tests run in a stable state.
- Data Setup/Teardown: Ensure each test starts in a clean data environment and that data is cleared after completion.
- Loading/Error State Display: Test that the UI correctly displays loading and error states to the user.
- Accessibility Testing: Use tools like Accessibility Scanner in UI tests to ensure app accessibility.



Esmaeel Moradi
@esmaeel-moradi700

Architect-Level Insights:



- **Trade-off between Custom Layout and Modifiers:** Always try to achieve the desired result first with existing Modifiers (like `graphicsLayer`, `animateContentSize`, `clip`). If Modifiers are insufficient, then resort to the Layout Composable. `graphicsLayer` is often a better choice for visual animations due to direct application of transformations on the GPU and not triggering recompositions.
- **Performance Bottlenecks:**
 - Over-Recomposition: Ensure that expensive computations (like `offsetFraction` calculations and transformations) only occur when the state truly changes (use of `derivedStateOf`).
 - Allocation: Avoid excessive memory allocation on every recomposition.
 - Measurement/Layout Passes: Custom Layouts can trigger multiple measurement and layout passes. Consider this and try to avoid unnecessary multi-pass layouts.
- **Accessibility:** Ensure your animated UI remains accessible to users with special needs. Use `contentDescription` and accessibility services correctly.
- **Separation of Concerns:** Keep complex UI and animation logic separate from business logic. The ViewModel should provide state and events, not manage animations.
- **Testing:** Testing animations can be challenging. Use Compose Test APIs to verify that animations start and end correctly. `ComposeTestRule.mainClock.advanceTimeBy()` is very useful for controlling animation time in tests.



Esmaeel Moradi
@esmaeel-moradi700

Supplementary Tests:



- Performance Tests:
 - Using Android Studio Profiler (CPU, Memory, Network) and Macrobenchmarks (for Jetpack Compose) to identify bottlenecks, ANRs, and Memory Leaks.
 - Testing startup time, UI jank, and battery consumption.
- Security Tests:
 - Especially for an application like a crypto wallet: Static Analysis (Lint, ktlint), Dynamic Analysis (Runtime checks), Penetration Testing (by external security teams).
 - Testing secure storage of information, authentication, and encryption.
- Usability Tests:
 - Testing with real users to gather feedback on the user experience.
- Regression Tests:
 - Rerunning all tests after every code change to ensure that new changes haven't broken existing features.



Esmaeel Moradi
@esmaeel-moradi700



Architect-Level Notes for Strategy Design:

- Quality at the Source: Encourage the team to write tests alongside the code. This helps write more testable and easily debuggable code.
- CI/CD Pipeline: All tests must be automatically run in the CI/CD pipeline. (More details in the Architect-Level question).
- Test Reporting and Metrics: Continuously monitor metrics like Code Coverage, Test Failure Rate, and test execution time.
- Manage "Flakiness": UI tests that intermittently fail (Flaky Tests) must be identified and fixed immediately, as they erode team confidence in the testing system.
- Use Appropriate Test Doubles: The choice between Mock, Stub, Spy, and Fake should be made carefully.
- Mock: For verifying interactions (e.g., `verify(mockObject).someMethod())`).
- Stub: For returning predefined values.
- Fake: A lightweight implementation of a real dependency (e.g., an in-memory database).
- Strategy for Third-Party Libraries: For large and complex libraries, you typically only test your interactions with them, not the library itself.



Esmaeel Moradi

@esmaeeil-moradi700

- 
- Domain-Driven Design (DDD) Awareness: If your architecture is based on DDD, tests should heavily cover the Domain and Application layers, as these layers contain critical business logic and change less frequently.

This comprehensive strategy allows you to confidently ensure the quality of your application, prevent costly bugs in later stages, and maintain a sustainable development process.



Esmaeeil Moradi
@esmaeeil-moradi700

How do you test Asynchronous Code, such as Coroutines or RxJava Streams?

Testing asynchronous code requires specific tools and approaches to gain precise control over timing and execution of operations and to deterministically validate results.

1. Testing Kotlin Coroutines:

- To test Coroutines, you need precise control over the Dispatcher that manages Coroutine execution. The `kotlinx-coroutines-test` library provides the necessary tools.
- Dependencies:

```
// app/build.gradle.kts
dependencies {
    // ...
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.0")
    // ...
}
```



Esmaeel Moradi
@esmaeeil-moradi700

Key Concepts:



- **TestDispatcher**: An interface implemented by **StandardTestDispatcher** and **UnconfinedTestDispatcher**.
 - **StandardTestDispatcher** (recommended): This allows you to manually advance Coroutine execution with methods like `advanceUntilIdle()`, `advanceTimeBy()`, and `runCurrent()`. This is the best choice for deterministic tests.
 - **UnconfinedTestDispatcher**: Executes Coroutines as soon as they are launched. Can be useful for very simple tests but offers less control.
- **runTest**: A test builder that provides a **CoroutineScope** with a **TestDispatcher** (defaulting to **StandardTestDispatcher**) and automatically resets the main Dispatcher after the test completes. It also automatically waits for all Coroutines within the **TestScope** to finish.
- **Dispatchers.setMain(testDispatcher)** / **Dispatchers.resetMain()**: For testing ViewModels or any class that uses **Dispatchers.Main** (like **viewModelScope** which defaults to **Dispatchers.Main**), you need to replace **Dispatchers.Main** with your custom **testDispatcher**. This should be done in **@Before** and **@After**.
- **advanceUntilIdle()**: Executes all pending Coroutines in the **TestDispatcher** until they have no more work to do. This method is very useful as it frees you from manually managing delays.



Esmaeel Moradi

@esmaeel-moradi700

Example: Testing UserViewModel (Revised for Coroutines Test):



```
// Same UserViewModel and UserRepository from the previous example

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.ExperimentalCoroutinesApi
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.test.StandardTestDispatcher
import kotlinx.coroutines.test.advanceUntilIdle
import kotlinx.coroutines.test.resetMain
import kotlinx.coroutines.test.runTest
import kotlinx.coroutines.test.setMain
import org.junit.After
import org.junit.Before
import org.junit.Test
import org.mockito.kotlin.mock
import org.mockito.kotlin.verify
import org.mockito.kotlin.whenever
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import kotlin.time.Duration.Companion.seconds

@OptIn(ExperimentalCoroutinesApi::class)
class UserViewModelTest {

    private lateinit var userRepository: UserRepository
    private lateinit var userViewModel: UserViewModel
    private val testDispatcher = StandardTestDispatcher() // 1. Create a test dispatcher

    @Before
    fun setup() {
        userRepository = mock()
        userViewModel = UserViewModel(userRepository)
        Dispatchers.setMain(testDispatcher) // 2. Set the main dispatcher
    }

    @After
    fun tearDown() {
        Dispatchers.resetMain() // 3. Reset the main dispatcher
    }

    @Test
    fun loadUser_success_updatesUserStateAndLoadingStatus() = runTest(timeout = 2.seconds) {
        // Given
        val expectedUser = User("1", "Test User", "test@example.com")
        // Use `coEvery` from MockK if you use MockK with coroutines
        whenever(userRepository.fetchUser("1")).thenReturn(expectedUser)
    }
}
```



Esmaeel Moradi
@esmaeel-moradi700



```
// When
userViewModel.loadUser("1")
assertTrue(userViewModel.isLoading.value) // Should be loading immediately
advanceUntilIdle() // 4. Advance time until all coroutines are idle

// Then
assertFalse(userViewModel.isLoading.value)
assertNull(userViewModel.errorMessage.value)
assertEquals(expectedUser, userViewModel.userState.value)
verify(userRepository).fetchUser("1")
}

@Test
fun loadUser_failure_updatesErrorMessageAndLoadingStatus() = runTest {
    // Given
    val errorMessage = "Network Error"
    whenever(userRepository.fetchUser("2")).thenThrow(RuntimeException(errorMessage))

    // When
    userViewModel.loadUser("2")
    assertTrue(userViewModel.isLoading.value)
    advanceUntilIdle() // Advance time to ensure coroutine finishes

    // Then
    assertFalse(userViewModel.isLoading.value)
    assertNull(userViewModel.userState.value)
    assertEquals(errorMessage, userViewModel.errorMessage.value)
    verify(userRepository).fetchUser("2")
}
```



Esmaeel Moradi
@esmaeel-moradi700



2. Testing RxJava Streams:

To test RxJava, we use TestScheduler and TestObserver (for Flowables, Observables, Singles, Maybes) or TestSubscriber (for Flowables).

Dependencies:

```
// app/build.gradle.kts
dependencies {
    // ...
    testImplementation("io.reactivex.rxjava3:rxjava:3.1.8")
    testImplementation("io.reactivex.rxjava3:rxkotlin:3.0.1")
    testImplementation("io.reactivex.rxjava3:rxandroid:3.0.2")
    testImplementation("io.reactivex.rxjava3:rxjava-debug:3.0.0") // Optional, for debugging
    // ...
}
```



Esmaeel Moradi
@esmaeeil-moradi700

Key Concepts:

- TestScheduler: Allows you to manually control time in RxJava streams (e.g., advanceTimeBy()).
- TestObserver / TestSubscriber: Objects that subscribe to your Rx stream and allow you to validate emitted values, errors, and completion status.
- RxJavaPlugins.setIoScheduler() / RxJavaPlugins.setComputationScheduler() / RxJavaPlugins.setNewThreadScheduler(): For replacing real Schedulers with your TestScheduler during testing.

Example: Testing a Simple RxJava Class:

```
// src/main/java/com/example/myapp/RxDataProcessor.kt
package com.example.myapp

import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.schedulers.Schedulers
import java.util.concurrent.TimeUnit

class RxDataProcessor {
    fun fetchAndProcessData(dataId: String): Observable<String> {
        return Observable.just("Raw Data for $dataId")
            .delay(500, TimeUnit.MILLISECONDS, Schedulers.io()) // Simulate network delay
            .map { it.uppercase() } // Process data
            .doOnNext { println("Processing $it") }
            .subscribeOn(Schedulers.io())
            .observeOn(Schedulers.computation())
    }
}
```



Esmaeel Moradi
@esmaeeil-moradi700

Testing RxDataProcessorTest:



```
// src/test/java/com/example/myapp/RxDataProcessorTest.kt
package com.example.myapp

import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.plugins.RxJavaPlugins
import io.reactivex.rxjava3.schedulers.TestScheduler
import org.junit.After
import org.junit.Before
import org.junit.Test
import org.junit.Assert.assertEquals
import java.util.concurrent.TimeUnit

class RxDataProcessorTest {

    private lateinit var rxDataProcessor: RxDataProcessor
    private lateinit var testScheduler: TestScheduler

    @Before
    fun setup() {
        rxDataProcessor = RxDataProcessor()
        testScheduler = TestScheduler()

        // Replace all Schedulers with our TestScheduler
        RxJavaPlugins.setIoSchedulerHandler { testScheduler }
        RxJavaPlugins.setComputationSchedulerHandler { testScheduler }
        RxJavaPlugins.setNewThreadSchedulerHandler { testScheduler }
    }

    @After
    fun tearDown() {
        // Reset Schedulers to default after each test
        RxJavaPlugins.reset()
    }

    @Test
    fun fetchAndProcessData_success_returnsUppercaseData() {
        // Given
        val dataId = "test_id"
        val testObserver = rxDataProcessor.fetchAndProcessData(dataId).test()

        // Initially, no data should be received as there's a delay
        testObserver.assertNoValues()
        testObserver.assertNotComplete()

        // When: Advance time by the simulated delay (500 ms)
        testScheduler.advanceTimeBy(500, TimeUnit.MILLISECONDS)

        // Then: Assert that the processed data is received and stream is completed
        testObserver.assertValue("RAW DATA FOR TEST_ID")
        testObserver.assertComplete()
        testObserver.assertNoErrors() // Ensure no errors occurred
    }
}
```



Esmaeel Moradi
@esmaeel-moradi700

```

@Test
fun fetchAndProcessData_multipleSubscribers_eachReceivesData() {
    val dataId = "another_id"
    val observable = rxDataProcessor.fetchAndProcessData(dataId)

    val observer1 = observable.test()
    val observer2 = observable.test()

    testScheduler.advanceTimeBy(500, TimeUnit.MILLISECONDS)

    observer1.assertValue("RAW DATA FOR ANOTHER_ID").assertComplete()
    observer2.assertValue("RAW DATA FOR ANOTHER_ID").assertComplete()
}

```

RxJava Testing Explanation:

- `test()`: A method from `RxJavaTest` that returns a `TestObserver`.
- `testObserver.assertNoValues()`, `assertValue()`, `assertComplete()`, `assertNoErrors()`: `TestObserver` methods for validating stream status.
- `testScheduler.advanceTimeBy()`: This method tells the `TestScheduler` to advance time, causing any delay or timer operations in the Rx stream to execute.
- Architect-Level Note: When testing asynchronous code, you must always be careful that all operations have truly completed to avoid "Flaky Tests". Using `advanceUntilIdle()` for Coroutines and `advanceTimeBy()` for RxJava are best practices.



Esmaeel Moradi
 @esmaeel-moradi700

How do you identify and fix common performance issues (like Memory Leaks, ANRs) using Android Studio Profiler?

Android Studio Profiler is a powerful suite of tools for diagnosing and analyzing performance issues in Android applications. These tools include the CPU Profiler, Memory Profiler, Network Profiler, and Energy Profiler.

1. Identifying and Fixing Memory Leaks:

How to Identify with Memory Profiler:

- **Goal:** A memory leak occurs when an object is held in memory, even though it's no longer needed and should have been garbage collected (GC). This leads to increased memory consumption and eventually an OutOfMemoryError and app crash.
- **Open Memory Profiler:**
 - a. Run your app on an emulator or device.
 - b. In Android Studio, open the Profiler tab.
 - c. Select Memory.



Esmaeel Moradi
@esmaeel-moradi700

- **Capture a Heap Dump:**



1. While the app is running, repeat the operation suspected of causing a memory leak several times (e.g., opening and closing an Activity/Fragment, or rotating the device to change configuration).
2. Click the Dump Java heap button (trash can icon with a + sign).
3. A Heap Dump of the current memory state will be captured.

- **Analyze the Heap Dump:**

1. In the Heap Dump, sort objects by Class Name or Instance Count.
 2. Look for objects you don't expect to be in memory (e.g., an Activity or Fragment that should have been destroyed but still exists) or whose count is unreasonably high.
 3. Right-click on the suspicious object and select Go to Instances.
 4. In the Instances window, right-click on the object and select Analyze references. This will show the "Path to GC Root," which is the chain of references preventing the object from being garbage collected.
- Tip: Typically, the root of a memory leak is a static reference, an anonymous inner class holding a reference to an Activity/Fragment, a Listener, or a Handler that wasn't properly deregistered.



Esmaeel Moradi
@esmaeel-moradi700

How to Fix Memory Leaks:



- **Eliminate Strong References to Context:** Especially Activity or Fragment Context. Use Application Context for long-lived dependencies.
- **Deregister Listeners and Broadcast Receivers:** Always deregister Listeners and Receivers in appropriate lifecycle methods (onDestroy, onStop, onPause).
- **Handlers and Runnables:** If using Handler and Runnable, ensure they are properly nulled or removeCallbacks is called to prevent referencing the Activity. Use WeakReference for Context inside a Handler.
- **Non-Static Inner Classes:** Use non-static Inner Classes (which implicitly hold a strong reference to the outer class) with caution, especially if their lifespan is longer than the outer class. Use static inner classes along with WeakReference.
- **Large Images:** Optimize images (scaling, compression) and load them with libraries like Glide/Coil/Picasso that handle memory management well.
- **LeakCanary:** This library is an excellent tool for automatically detecting memory leaks during development and notifying you when a leak occurs, showing the Path to GC Root.



Esmaeel Moradi

@esmaeeil-moradi700

2. Identifying and Fixing ANRs (Application Not Responding):



ANR Goal: An ANR error occurs when the application's main thread (UI Thread) is blocked for too long (typically more than 5 seconds for user input or 10 seconds for a Broadcast Receiver) and cannot respond to user events. This leads to the application becoming unresponsive and displaying the "Application Not Responding" dialog.



Esmaeeil Moradi
@esmaeeil-moradi700

How to Identify with CPU Profiler:



- **Open CPU Profiler:**
 - a. Run the app.
 - b. In Profiler, select CPU.
 - c. Start a recording (the Record button).
- **Simulate ANR:** Perform an operation suspected of blocking the UI Thread (e.g., an infinite loop, disk or network operation on the Main Thread).
- **Analyze the Recording:**
 - a. In the CPU Usage graph, look for spikes in CPU usage that align with your app's unresponsiveness.
 - b. In Thread Activity, find the Main Thread (or UI Thread) and zoom in.
 - c. Look for sections where the Main Thread was executing a heavy operation for an extended period (e.g., dark red coloring in the graph).
 - d. Use the Call Chart, Flame Chart, Top Down, Bottom Up panels to analyze the Call Stack and identify which function or method consumed the most time on the Main Thread. Typically, an I/O operation (network, disk) or a heavy computational loop is the cause of the ANR.



Esmaeel Moradi
@esmaeel-moradi700

How to Fix ANRs:



- **Perform Heavy Operations Off the Main Thread:**
 - Coroutines: The best and most modern solution. Use Dispatchers.IO for I/O operations and Dispatchers.Default for heavy computational operations. Display results on the Main Dispatcher.
 - RxJava: Use `subscribeOn(Schedulers.io())` and `observeOn(AndroidSchedulers.mainThread())`.
 - ThreadPools / Executors: You can use custom Executors for background operations.
 - AsyncTask (deprecated): Do not use.
- **Optimize Layout Performance:**
 - Reduce Overdraw: Avoid excessive Overdraw in the UI (by removing unnecessary Views, using appropriate background colors).
 - Complex Layout Hierarchies: Avoid overly deep or complex layout hierarchies. ConstraintLayout or Jetpack Compose can help.
 - Expensive `onDraw()`: Avoid performing heavy operations in the `onDraw()` method of custom Views.
- **Early App Startup:** Delay heavy operations in `Application.onCreate()` or `Activity.onCreate()` or move them to the background.
- **StrictMode:** Use StrictMode in Debug mode to detect unwanted operations on the Main Thread (like disk or network access).



Esmaeel Moradi
@esmaeel-moradi700

Architect-Level Notes for Performance:

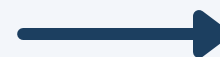


- **Continuous Profiling:** Performance should be part of the development cycle, not just at the end.
- **Macrobenchmarks (for Compose):** This library allows you to automatically test UI performance (Jank) and ensure frames are rendered consistently.
- **AOSP Tools (e.g., perfetto):** For deeper analysis of system performance and tracing lower-level Call Stacks, tools like perfetto (on Linux/macOS systems) can be used.
- **Performance Monitoring in Production:** Use tools like Firebase Performance Monitoring or Crashlytics (for ANRs and crashes) to collect performance data from real users and proactively identify issues.
- **Design for Performance:** Design the application's architecture with performance in mind from the outset (e.g., using layered architecture, proper use of Coroutines/RxJava).

With these tools and approaches, you can effectively identify and fix performance issues in Android applications, ensuring a smooth and stable experience for end-users.



Esmaeel Moradi
@esmaeel-moradi700



Architect Level

This section is in Part 2



Esmaeeil Moradi

@esmaeeil-moradi700

✨ These comprehensive answers aim to equip you with the knowledge required for Architect-level and Senior-level interviews, and to make informed architectural decisions in your real-world projects. Good luck. ✨



Esmaeeil Moradi

@esmaeeil-moradi700